Multilingual Speech Recognition Under Compute Constraints: A Routed Whisper and Indic ASR Pipeline

Author: Sagandeep Kaur **Project:** NPPE-2 Multilingual Speech Recognition Challenge

Abstract

This paper presents an end-to-end multilingual automatic speech recognition (ASR) system developed for the NPPE-2 Multilingual Speech Recognition challenge. The task required transcribing short audio clips from multiple languages into verbatim text, with submissions evaluated using Word Error Rate (WER). The available dataset contained 2,000 labeled training clips and 100 unlabeled test clips. The training set was approximately balanced across English written in Latin script, Hindi or related North Indian speech written in Devanagari script, and Tamil speech written in Tamil script.

Several systems were explored in the repository: zero-shot Whisper baselines, beam-search inference, parameter-efficient Whisper fine-tuning with LoRA-style adapters, checkpoint-based inference from Hugging Face models, and a final hybrid routing pipeline. The strongest design direction was a practical routed system: a lightweight Whisper model first performs language identification; English speech is routed to Whisper large-v3-turbo, while Indic-language speech is routed to AI4Bharat IndicConformer-600M with manual routing corrections such as Urdu-to-Hindi and Malayalam-to-Tamil. The study finds that, under limited compute and hidden-test evaluation, robust inference, language-aware routing, and conservative text normalization are more reliable than rushed fine-tuning on a small multilingual dataset.

1. Introduction

Automatic speech recognition converts acoustic speech signals into text. Modern ASR systems are often built from large pretrained encoder-decoder or transducer models, which can generalize across speakers, recording conditions, and languages. Multilingual ASR is harder than monolingual ASR because the system must handle different phoneme inventories, scripts, vocabulary distributions, code-switching patterns, and language-specific orthographic conventions.

The NPPE-2 Multilingual Speech Recognition challenge required participants to build a system that generated transcripts for multilingual audio clips. The competition used WER as the evaluation metric, so the objective was not merely to identify the language or produce semantically similar text, but to match the reference transcript at the word level.

The work in this repository is representative of a practical competition setting. The development process was constrained by time, Kaggle/Colab GPU availability, model download limits, and the absence of public labels for the test set. Because of these constraints, the project evolved from simple baselines toward a hybrid system that used pretrained models more strategically instead of relying only on fine-tuning.

2. Dataset

The dataset archive `data/multilingual-speech-recognition.zip` contains the official competition data:

Split	Rows	Audio files	Columns
Train	2,000	2,000	`id`, `audio`, `text`
Test	100	100	`id`, `audio`
Sample submission	100	-	`audio`, `text`

The training transcripts contain 1,997 unique text strings and no empty labels. The average transcript length is 17.94 words, with a minimum of 1 word and a maximum of 62 words. All audio files are mono WAV files. The training clips have a mean duration of 7.70 seconds and a median duration of 5.73 seconds; the shortest clip is 1.02 seconds and the longest is 24.19 seconds. The test clips have similar duration statistics, with a mean duration of 7.93 seconds and a median duration of 5.50 seconds.

Script-level inspection of the training labels gives the following distribution:

Dominant script	Count
Latin	998
Devanagari	508
Tamil	494

This distribution shaped the modeling strategy. A single multilingual model could be used for all examples, but the dataset composition suggested a natural routing structure: English/Latin-script examples, Hindi-family/Devanagari examples, and Tamil-script examples.

3. Evaluation Metric

The competition evaluates submissions using Word Error Rate:

```
WER = (Substitutions + Deletions + Insertions) / Number of words in reference
```

Lower WER is better, and a WER of 0 indicates a perfect transcript. WER is sensitive to missing words, hallucinated words, casing if the scorer is case-sensitive, punctuation if punctuation is not normalized by the scorer, script mismatch, and transliteration mismatch. For example, a Hindi utterance transcribed in Latin characters may be semantically recognizable but still receive a poor WER against a Devanagari reference.

4. Repository Audit

The repository contains multiple generations of experiments.

The earliest notebooks, such as `dlp-nppe-2 nb 1.ipynb`, used `openai/whisper-base` with CPU inference and basic text cleaning. This was a useful correctness baseline but likely too weak for a competitive score.

The notebook `nb cpu 0.8.ipynb` moved to `openai/whisper-small`, added beam search with `num_beams=5`, and applied lowercase/punctuation cleanup. The local notes identify this as the strongest simple baseline among the early notebooks.

The notebooks `dlp-nppe-2-clean-fixed-v2.ipynb`, `dlp-nppe-2-fixed-whisper-lora.ipynb`, `dlp-nppe-2-from gpt.ipynb`, and `full-patched-notebook-gpt-pro penalities.ipynb` explored fine-tuning `openai/whisper-small` with a PEFT/LoRA-style wrapper. These notebooks include preprocessing, train/validation splitting, WER/CER metric computation, and patched wrappers to handle Whisper's `input_features` interface. They also show issues common in rushed ASR fine-tuning: validation/inference mismatch, greedy validation decoding, potential overfitting, and risky post-processing such as uppercasing English outputs.

The files `nppe2_best_of_both_safe.ipynb` and `nppe2_colab_hybrid_0p1.ipynb` represent the later practical phase. The first keeps the reliable Whisper-small beam-search path and adds optional fine-tuning only when it can be validated. The second implements a higher-upside hybrid system using faster-whisper language identification, Whisper large-v3-turbo for English, and AI4Bharat IndicConformer for Indic routes.

The final notebook `23F2003511_DLP_26T1_NPPE2.ipynb`, present both at the workspace root and inside `Multilingual-Speech-Recognition/`, follows the hybrid design. It includes path handling for Kaggle and Colab, Hugging Face token handling, language detection, routed inference, validation scaffolding, caching, and final submission generation.

There are many generated submission files. Their contents show the same experimental progression: blank/sample submissions, Whisper-only outputs, checkpoint outputs, Indic-specific outputs, ensemble attempts, encoding-corrupted variants, and final hybrid submissions. Several CSVs contain all 100 test rows, but some have one empty transcript or encoding artifacts. This reinforces the importance of output validation before submission.

5. Methodology

5.1 Audio and Metadata Handling

The pipeline reads `train.csv` and `test.csv`, resolves each audio filename to the correct split directory, and creates absolute paths for inference. The code includes fallback path candidates because the same notebook was intended to run both in Kaggle and Colab, where the extracted directory layout may differ.

For local validation, the training set is split into train and validation partitions using a fixed random seed. Since language labels are not provided directly, the code derives a script bucket from the transcript using Unicode character names. Rare scripts are collapsed into a `RARE` bucket before stratification. This preserves approximate language/script balance in the validation subset.

5.2 Text Normalization

Text normalization is deliberately conservative. The helper functions collapse whitespace, remove underscores, and remove punctuation while preserving Latin, Devanagari, and Tamil characters. English predictions are lowercased because prior submissions and project notes suggested that lowercase English improved compatibility with the expected transcript style. Non-English scripts are not lowercased or transliterated because script preservation is central to WER.

This choice is important. Aggressive cleanup can reduce punctuation-related errors, but it can also damage Indic Unicode text if encoding is mishandled or if the regex excludes the target script. Some generated CSVs in the repository show mojibake, which would severely harm WER even if the original prediction was good.

5.3 Baseline Whisper Inference

The first modeling family uses Whisper through Hugging Face Transformers. Whisper is a multilingual encoder-decoder ASR model trained on large-scale weak supervision. The project tried `openai/whisper-base` and `openai/whisper-small`, with the stronger baseline using beam search (`num_beams=5`) and `max_new_tokens=255`.

The main advantage of this baseline is simplicity. It does not require training, works across languages, and can run in CPU or GPU environments. Its weaknesses are model capacity and language/script errors, especially for Indic speech where Whisper may produce phonetically plausible but script-incorrect text.

5.4 Parameter-Efficient Whisper Fine-Tuning

Several notebooks attempted to fine-tune Whisper-small with LoRA-style parameter-efficient adaptation. LoRA freezes most pretrained weights and trains low-rank adapter matrices, reducing training memory and making adaptation feasible on limited GPUs. In this repository, the fine-tuning path included:

- `openai/whisper-small` as the base model.
- Feature extraction with `WhisperProcessor`.
- Train/validation splits stratified by script and duration bucket.
- WER and CER computation using `evaluate`/`jiwer`.
- A custom `WhisperTuner` wrapper to handle Whisper's speech-specific `input_features`.
- Training settings around batch size 8, gradient accumulation 2, learning rate near `1e-4`, and multi-epoch training.

Fine-tuning was not clearly superior in the repository evidence. The notes identify several reasons: small multilingual data, possible train/test mismatch, greedy validation decoding, mismatch between validation normalization and submission normalization, and overfitting or language drift. Saved Hugging Face checkpoints such as `sagandeep/whisper-medium-hindi-lora-v2`, `sagandeep/whisper-medium-checkpoint-675`, and `sagandeep/whisper-medium-checkpoint-500` were considered for inference, but the later notebooks advise using them only if validation proves they beat the pretrained route.

5.5 Routed Hybrid Inference

The most developed system is the routed hybrid pipeline. It has three stages.

First, a lightweight faster-whisper model performs language identification. Each audio file receives a predicted language code and confidence score. faster-whisper is useful here because it reimplements Whisper inference with CTranslate2 and is designed for faster, lower-memory inference.

Second, predicted language codes are mapped into route groups:

Predicted language	Route
`en`	English Whisper route
`hi`, `ur`, `mr`, `ne`, `mai`, `doi`, `sa`	Hindi/Devanagari Indic route
`ta`, `ml`	Tamil Indic route
Other	Whisper fallback route

The manual mappings are motivated by observed dataset behavior and competition advice: some clips detected as Urdu are expected to be transcribed in Hindi/Devanagari, and some Malayalam detections are better handled through Tamil routing for this dataset.

Third, each route uses the model most appropriate for the expected script:

- English route: `large-v3-turbo` via faster-whisper, forced language `en`, beam size 5.
- Hindi route: `ai4bharat/indic-conformer-600m-multilingual`, language code `hi`, RNNT decoding.
- Tamil route: `ai4bharat/indic-conformer-600m-multilingual`, language code `ta`, RNNT decoding.
- Fallback route: Whisper without a forced language.

The IndicConformer model card describes the model as a 600M-parameter multilingual Conformer-based hybrid CTC + RNNT ASR model supporting the 22 official Indian languages. This makes it a natural complement to Whisper for Devanagari and Tamil-script outputs.

6. Experiments

The repository records the following experiment families:

Experiment family	Main files	Key idea	Observed risk
Whisper base CPU	`dlp-nppe-2 nb 1.ipynb`	Simple zero-shot baseline	Weak model and greedy decoding
Whisper small beam search	`nb cpu 0.8.ipynb`	Stronger no-training baseline	Still language/script errors
Whisper-small LoRA	`dlp-nppe-2-clean-fixed-v2.ipynb`, `full-patched-notebook-gpt-pro penalities.ipynb`	Adapt Whisper to the training set	Overfitting, wrapper complexity, validation mismatch

Experiment family	Main files	Key idea	Observed risk
Checkpoint inference	`submission-checkpoint-675.csv`, related files	Use saved fine-tuned checkpoints	Mixed script behavior and uncertain validation gain
Indic-only or Indic-heavy outputs	`submission-indic.csv`, `submission hybrid codex.csv`	Improve Indic-language transcription	May mis-handle English route
Hybrid routing	`nppe2_colab_hybrid_0p1.ipynb`, `23F2003511_DLP_26T1_NPPE2.ipynb`	Use language ID and route to specialized ASR models	Depends on language ID quality and gated model access

Because the hidden test labels are not included in the repository, exact test WER cannot be recomputed locally. Some notebooks contain validation scaffolding, but the final validation results are not fully recorded in the saved notebook outputs. Therefore, this paper treats the repository as an applied system-development study rather than a fully reproducible benchmark report with final hidden-test scores.

7. Results and Analysis

The main result is qualitative but practically significant: the best-supported direction is not a single fine-tuned model, but language-aware model routing.

The early Whisper-small beam-search baseline is strong because it avoids training instability and uses a robust pretrained multilingual model. However, the sample submissions show that generic Whisper outputs can select the wrong script for Indic speech or produce approximate phonetic text. Since WER compares words exactly, script mismatch is a major failure mode.

Fine-tuning was an attractive option, especially with LoRA, because the training set was domain-specific and small enough for fast iteration. In practice, the repository evidence suggests that fine-tuning introduced additional risks. The training set has only 2,000 examples spread across at least three major script groups, so a single adapter may improve one group while hurting another. The notebooks also show that validation configuration matters: a checkpoint selected with greedy decoding and raw WER may not be the best checkpoint under beam-search inference and submission cleaning.

The hybrid route directly addresses the major observed error source. Whisper is retained for English, where it is robust and produces natural Latin-script transcripts. IndicConformer is used for Hindi and Tamil routes, where script-specific ASR is more important. The routing layer also provides a place for competition-specific corrections, such as mapping Urdu detections to Hindi and Malayalam detections to Tamil.

The generated submissions reveal practical output issues:

- Some files have one empty `text` entry.

- Some CSVs contain mojibake from incorrect encoding handling.
- Some outputs uppercase English text, which may hurt if scoring is case-sensitive.
- Some models produce the wrong script even when the spoken content is recognizable.

These issues are not modeling details alone; they are end-to-end ASR system issues. For WER competitions, final CSV validation is part of the model pipeline.

8. Limitations

This work has several limitations.

First, the repository does not contain hidden test references or official leaderboard scores for every submission. As a result, the final WER of the hybrid system cannot be independently verified from local files alone.

Second, language identification is itself imperfect. A wrong route can send an audio file to a model that produces the wrong script or language. The manual route corrections reduce this risk for known dataset patterns, but they are not guaranteed to generalize beyond this competition.

Third, the fine-tuning experiments were constrained by GPU availability and time. A more rigorous study would evaluate all checkpoints under identical decoding and normalization settings, report per-script WER, and tune route-specific thresholds.

Fourth, the current hybrid system depends on external Hugging Face model access, including a gated AI4Bharat model. This affects reproducibility unless access tokens and model permissions are configured correctly.

9. Future Work

Several improvements follow naturally from the repository findings.

First, validation should report WER separately for Latin, Devanagari, and Tamil buckets. Per-script WER would make it clear whether a change improves one language group while degrading another.

Second, route selection could use confidence thresholds instead of hard language-code rules. For low-confidence language ID, the system could run both Whisper and IndicConformer and select a transcript using script detection, language-model scoring, or validation-derived heuristics.

Third, fine-tuning should be revisited with stricter experiment control. The same decoding settings, text normalization, and WER computation should be used for validation and final inference. Separate adapters for English, Hindi, and Tamil may be safer than a single multilingual adapter.

Fourth, CSV validation should be automated. Before submission, a script should verify row count, column names, UTF-8 encoding, no empty transcripts, no duplicate audio IDs, and script compatibility with the predicted route.

10. Conclusion

This project demonstrates that multilingual ASR performance in a constrained competition setting depends as much on system design as on model choice. A simple Whisper-small beam-search

baseline provides a reliable starting point, but the dataset's mixture of Latin, Devanagari, and Tamil scripts creates failure modes that a single generic model may not handle well. LoRA-style fine-tuning is feasible but fragile when training data is small, multilingual, and evaluated through a hidden WER metric.

The most defensible architecture in the repository is a routed hybrid pipeline. It uses Whisper for language identification and English transcription, AI4Bharat IndicConformer for Hindi and Tamil routes, conservative Unicode-preserving cleanup, and final CSV generation. The central lesson is that for multilingual ASR, especially under time and compute constraints, language-aware routing and careful post-processing can be more valuable than rushed fine-tuning.

References

1. Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. "Robust Speech Recognition via Large-Scale Weak Supervision." arXiv:2212.04356, 2022. https://arxiv.org/abs/2212.04356
2. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models." arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685
3. AI4Bharat. "IndicConformer-600M-Multi." Hugging Face model card. https://huggingface.co/ai4bharat/indic-conformer-600m-multilingual
4. SYSTRAN. "faster-whisper: Faster Whisper transcription with CTranslate2." GitHub repository. https://github.com/SYSTRAN/faster-whisper
5. Tahir Javed, Sumanth Doddapaneni, Abhigyan Raman, Kaushal Santosh Bhogale, Gowtham Ramesh, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh M. Khapra. "Towards Building ASR Systems for the Next Billion Users." arXiv:2111.03945, 2021. https://arxiv.org/abs/2111.03945